

SIH26153 — Counterfactual Cyber World Model

AI-Based Network Attack Forecasting from Network Traffic Data (NTRO)

1. Overview

This system is a temporal Network World Model that learns $P(S_{t+1} | S_t)$, forecasts multiple future attack states (K-step rollout), maps them to MITRE ATT&CK stages, builds a predictive attack graph, and runs counterfactual defensive simulation to recommend the action that most reduces predicted future risk. It runs **fully offline** — no cloud or external inference API.

2. Prerequisites

Python 3.10 or newer (tested on 3.13). No cloud account or Npcap required.

Optional: to ingest .pcap files, install scapy (pip install scapy).

3. Install dependencies

```
python -m venv venv
venv\Scripts\activate // Windows
source venv/bin/activate // macOS / Linux

pip install -r requirements.txt
```

requirements.txt installs flask, gunicorn, numpy, scikit-learn, torch (CPU build is fine), shap, and pytest.

4. Run the dashboard

Run from the project root (the directory that contains run.py):

```
cd SIH26153-AI-Network-Attack-Forecasting/main
python run.py
```

Then open **http://localhost:5000** in your browser. On first request the LSTM World Model is trained on synthetic data and cached for the process lifetime.

Other run modes

Command	Description
python run.py	Train World Model, then start dashboard
python run.py --pipeline-only	Train + evaluate + forecast, print JSON report, exit
python run.py --no-pipeline	Dashboard only (trains lazily on first request)
python run.py --port 8080	Serve on a specific port

5. Dashboard pages (8)

- **/dashboard** — Live / uploaded traffic — current risk + stage
- **/radar** — 10-minute prediction radar (future risk timeline)
- **/graph** — Predictive attack graph (current + predicted states)

- **/counterfactual** — What-if defensive simulation + recommended action
- **/stages** — Attack stage timeline + MITRE trajectory
- **/explainability** — SHAP / top contributing feature explanations
- **/evaluation** — World Model vs baseline metrics
- **/scenarios** — Ad-hoc scenario / data explorer

6. Example forecast output

For the most recent observed window the system produces something like:

Step	Predicted Stage	Risk
Current	Command and Control	93%
t+1	Command and Control	93%
t+2	Exfiltration	<1%
t+3	Exfiltration	<1%
t+4	Command and Control	<1%
t+5	Exfiltration	<1%

Values shown are from an actual run and vary with the data; the dashboard always shows live computed values. All probabilities are real model outputs — nothing is fabricated.

7. Counterfactual defensive simulation (example)

For the same window the system simulates each defensive action through the World Model and compares the predicted future risk:

Action	Predicted Peak Risk
No Action	93%
Block Source	41%
Block Dest. Port	41%
Isolate Host	41%
Terminate Flow	41%
Restrict Path	41%
Recommended Action	Block Source

Each action modifies the simulated network state, the World Model runs a K-step rollout, and the learned risk head reads the resulting trajectory. The recommended action is the one that minimizes predicted near-term (peak) attack risk.

8. Run the tests

```
cd SIH26153-AI-Network-Attack-Forecasting/main
python -m pytest tests/ -v
```

The suite (61 tests) covers feature extraction, NetworkState construction, sequence generation, World Model training/prediction, K-step rollout, MITRE mapping, counterfactual actions, risk comparison, explainability, API, dashboard routes, and a full end-to-end pipeline.

9. Configuration (optional)

Copy `.env.example` to `.env` to customise:

Variable	Default	Purpose
PORT	5000	Web server port
FLASK_DEBUG	0	Enable Flask debug / hot reload
NW_WORLD_MODEL	lstm	Lstm or linear
NW_EPOCHS	30	Training epochs
NW_EXPLAIN	1	Enable SHAP explainability

10. Ingesting your own data

Dataset adapters exist in `netwatch/ingestion/datasets/adapters.py` for CIC-IDS2017/2018, CTU-13, UNSW-NB15, and CICIoT2023. They convert each source schema into the common `PacketRecord` / `NetworkState` schema. PCAP, flow CSV, and JSONL packet streams are supported via `netwatch/ingestion/parser.py`. Load a file through the matching adapter and re-run the pipeline — the World Model is dataset-agnostic.

11. Further reading

- `README.md`
- `docs/ARCHITECTURE.md`
- `docs/MODEL_CARD.md`
- `docs/EXPERIMENTS.md`
- `PROJECT_AUDIT.md`
- `REMOVED_COMPONENTS.md`
- `FINAL_AUDIT.md`